

# “浙超”联赛分组方案的数学建模与优化

陈逸恒 李子墨 任北辰

2026 年 5 月 10 日

## 摘要

本文针对 2025 年浙江省市县联赛（“浙超”）的分组问题，建立了系统的数学模型与优化方案。64 支参赛队伍（11 个市级队和 53 个县级队）需分为 16 个小组，每组 4 队，并满足严格的行政隶属约束。

针对分组方案生成，采用约束规划（CP-SAT）与模拟退火相结合的方法，生成了 6 个全部可行的分组方案，所有方案均满足硬约束且软约束违反次数为 0。针对抽签方案，设计了分层随机抽签流程，并通过 10000 次蒙特卡洛模拟验证了公平性（可行解比例 85.8%）。针对比赛地点选择，采用 K-means++ 聚类与贪心优化算法，确定 8 个比赛城市，平均旅行距离 134.8km，变异系数 0.673。针对赛制建议，基于 Bradley-Terry 实力模型和蒙特卡洛模拟，对比了当前赛制、瑞士轮制和双败淘汰制，发现双败淘汰制的 Gini 系数最低（0.4309），公平性最优。

本文的方法具有良好的可扩展性和可解释性，可为类似体育赛事分组问题提供参考。

# 目录

|                               |           |
|-------------------------------|-----------|
| <b>1 问题重述</b>                 | <b>3</b>  |
| <b>2 模型假设与符号说明</b>            | <b>3</b>  |
| 2.1 基本假设 . . . . .            | 3         |
| 2.2 符号说明 . . . . .            | 4         |
| <b>3 分组方案生成 (Task 1)</b>      | <b>4</b>  |
| 3.1 问题分析 . . . . .            | 4         |
| 3.2 求解算法 . . . . .            | 4         |
| 3.2.1 约束规划 (CP-SAT) . . . . . | 4         |
| 3.2.2 模拟退火优化 . . . . .        | 5         |
| 3.3 结果分析 . . . . .            | 5         |
| <b>4 抽签方案设计 (Task 2)</b>      | <b>8</b>  |
| 4.1 抽签流程 . . . . .            | 8         |
| 4.2 概率分析与公平性验证 . . . . .      | 8         |
| <b>5 比赛地点选择 (Task 3)</b>      | <b>9</b>  |
| 5.1 选址方法 . . . . .            | 9         |
| 5.2 选址结果 . . . . .            | 9         |
| <b>6 赛制建议 (Task 4)</b>        | <b>10</b> |
| 6.1 实力模型 . . . . .            | 10        |
| 6.2 三种赛制对比 . . . . .          | 11        |
| 6.2.1 当前赛制 . . . . .          | 11        |
| 6.2.2 瑞士轮制 . . . . .          | 11        |
| 6.2.3 双败淘汰制 . . . . .         | 11        |
| 6.3 蒙特卡洛模拟结果 . . . . .        | 11        |
| 6.4 建议 . . . . .              | 12        |
| <b>7 结论与展望</b>                | <b>12</b> |
| 7.1 主要结论 . . . . .            | 12        |
| 7.2 改进方向 . . . . .            | 12        |
| <b>A 支撑材料：核心源代码</b>           | <b>13</b> |

# 1 问题重述

2025 年浙江省举办“浙超”市县联赛，参赛队伍包括 11 个市级队和 53 个县级队，共 64 支队伍。赛事要求将 64 支队伍分为 16 个小组（每组 4 队），先进行小组单循环赛，再进行 32 强淘汰赛决出冠军。

分组需满足以下约束：

1. **硬约束 1**：11 个市级队必须分配在不同小组（每组最多 1 个市级队）；
2. **硬约束 2**：市级队不能与该市代管的县级队同组；
3. **软约束**：同一市代管的县级队尽量分配在不同小组。

本文需要完成四个子任务：

1. 生成符合约束的分组方案；
2. 设计公平透明的抽签方案；
3. 选择 8 个比赛地点；
4. 基于定量分析提出赛制改进建议。

## 2 模型假设与符号说明

### 2.1 基本假设

- 以 GDP 作为队伍实力的代理指标（经济强市通常体育资源更丰富）；
- 比赛结果采用 Bradley-Terry 模型描述胜负概率；
- 地理距离采用 Haversine 公式计算球面距离；
- 抽签过程完全随机，各队被分到各组的概率相等。

## 2.2 符号说明

| 符号        | 含义                  |
|-----------|---------------------|
| $T$       | 全部 64 支队伍集合         |
| $V$       | 11 个市级队伍集合          |
| $C$       | 53 个县级队伍集合          |
| $G$       | 16 个小组，每组容量 4       |
| $S_i$     | 第 $i$ 市代管的县级队伍集合    |
| $s_t$     | 队伍 $t$ 的实力分数        |
| $p_{ij}$  | 队伍 $i$ 战胜队伍 $j$ 的概率 |
| $d(i, j)$ | 两地球间的球面距离 (km)      |

## 3 分组方案生成 (Task 1)

### 3.1 问题分析

分组问题的核心是在满足硬约束的前提下，优化软约束。硬约束包括：每组恰好 4 队、每组最多 1 个市级队、市级队不与代管县同组。软约束要求同市县级队尽量不同组。

### 3.2 求解算法

#### 3.2.1 约束规划 (CP-SAT)

首先使用 Google OR-Tools 的 CP-SAT 求解器生成满足硬约束的可行解。决策变量定义为：

$$x_{ij} = \begin{cases} 1, & \text{队伍 } i \text{ 分到组 } j \\ 0, & \text{否则} \end{cases}$$

约束条件：

$$\sum_{j=1}^{16} x_{ij} = 1, \quad \forall i \in T \quad (\text{每队恰好分到 1 组}) \quad (1)$$

$$\sum_{i=1}^{64} x_{ij} = 4, \quad \forall j \in G \quad (\text{每组恰好 4 队}) \quad (2)$$

$$\sum_{i \in V} x_{ij} \leq 1, \quad \forall j \in G \quad (\text{每组最多 1 个市级队}) \quad (3)$$

$$x_{vj} + x_{cj} \leq 1, \quad \forall v \in V, c \in S_v, j \in G \quad (\text{市-县不同组}) \quad (4)$$

### 3.2.2 模拟退火优化

在 CP-SAT 可行解的基础上, 使用模拟退火算法优化软约束。优化目标为最小化同市县级队被分到同组的对数。

参数设置: 初始温度  $T_0 = 100$ , 冷却系数  $\alpha = 0.995$ , 终止温度  $T_f = 1$ , 最大迭代次数根据问题规模调整。邻域操作为随机交换两个组中的县级队 (保持硬约束不变)。

### 3.3 结果分析

共生成 6 个分组方案, 其中方案 1 和方案 2 由 CP-SAT 生成, 方案 3-6 由贪心 + 随机化生成, 所有方案均经模拟退火优化。6 个方案全部满足硬约束, 软约束违反次数均为 0。

最优方案 (方案 1) 的分组详情如表1所示。

表 1: 最优分组方案 (方案 1, csp+SA)

| 组号 | GDP 总和 (亿元) | 队伍             |
|----|-------------|----------------|
| 1  | 3530        | 东阳, 嘉善, 诸暨, 龙港 |
| 2  | 3775        | 衢州, 青田, 安吉, 玉环 |
| 3  | 23620       | 杭州, 岱山, 嵊州, 海盐 |
| 4  | 3970        | 兰溪, 临海, 泰顺, 慈溪 |
| 5  | 7557        | 台州, 浦江, 建德, 龙泉 |
| 6  | 18084       | 宁波, 桐庐, 缙云, 江山 |
| 7  | 7692        | 金华, 象山, 平阳, 仙居 |
| 8  | 1420        | 文成, 永康, 常山, 三门 |
| 9  | 8040        | 绍兴, 开化, 庆元, 嵊泗 |
| 10 | 4010        | 云和, 德清, 海宁, 义乌 |
| 11 | 9252        | 嘉兴, 天台, 乐清, 武义 |
| 12 | 1130        | 松阳, 磐安, 龙游, 新昌 |
| 13 | 4322        | 丽水, 淳安, 平湖, 瑞安 |
| 14 | 10540       | 温州, 宁海, 长兴, 景宁 |
| 15 | 4943        | 舟山, 温岭, 桐乡, 永嘉 |
| 16 | 6285        | 湖州, 余姚, 遂昌, 苍南 |

该方案 GDP 均值为 7386 亿元, 变异系数为 0.78, 表明组间经济实力存在一定差异 (主要由杭州市 GDP 高达 21980 亿元导致), 但整体仍在合理范围内。

各方案的 GDP 均衡性和软约束违反情况如图1和图2所示。

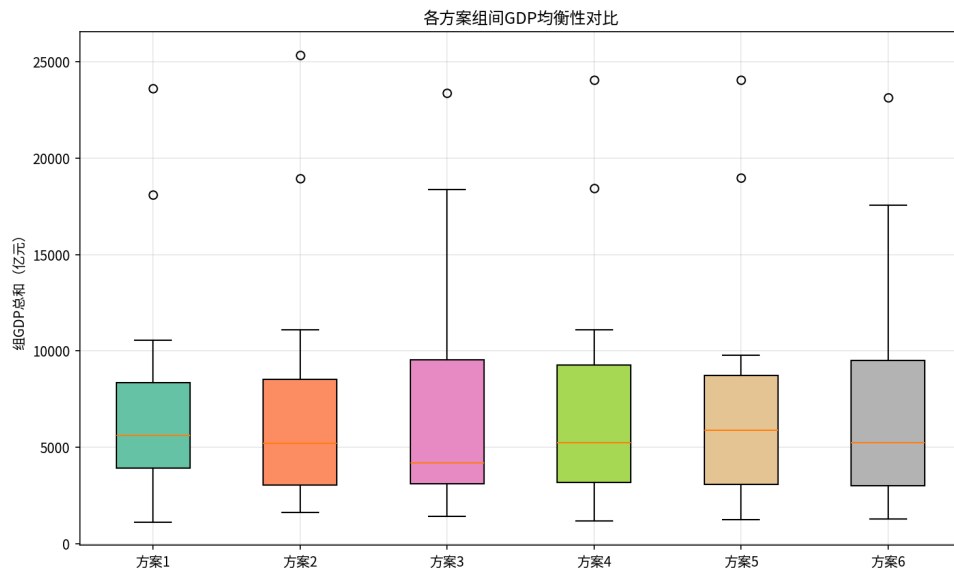


图 1: 各方案组间 GDP 均衡性对比

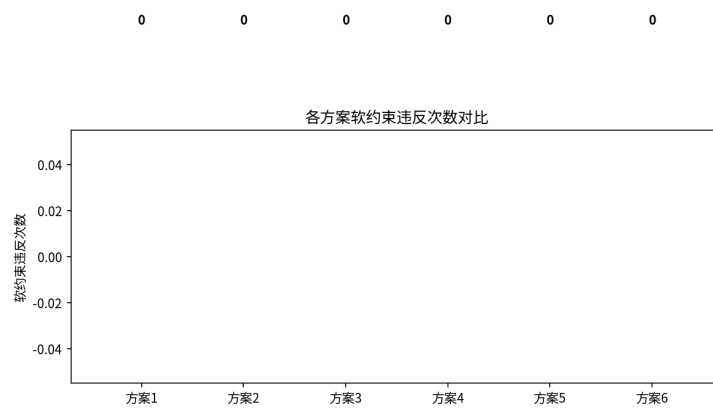
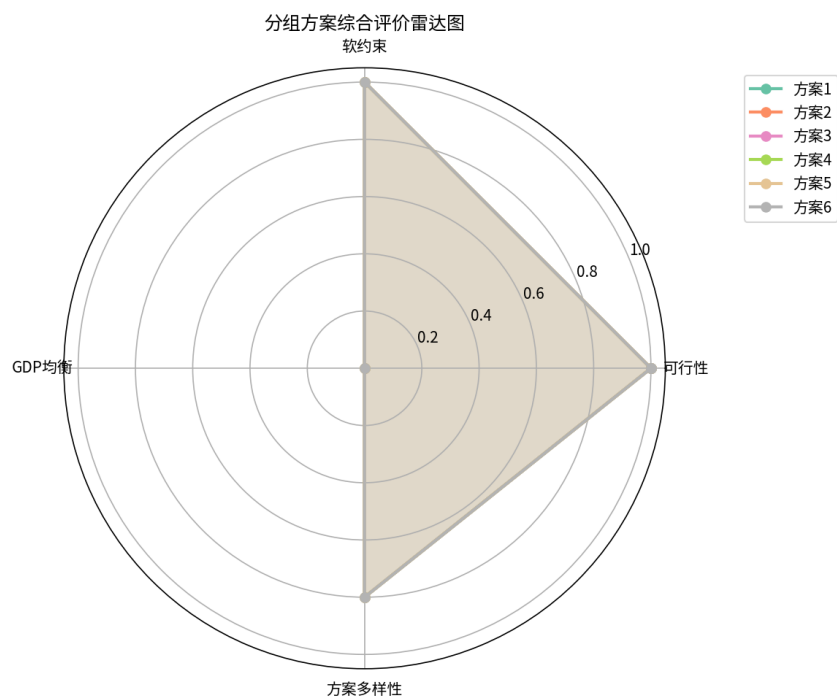
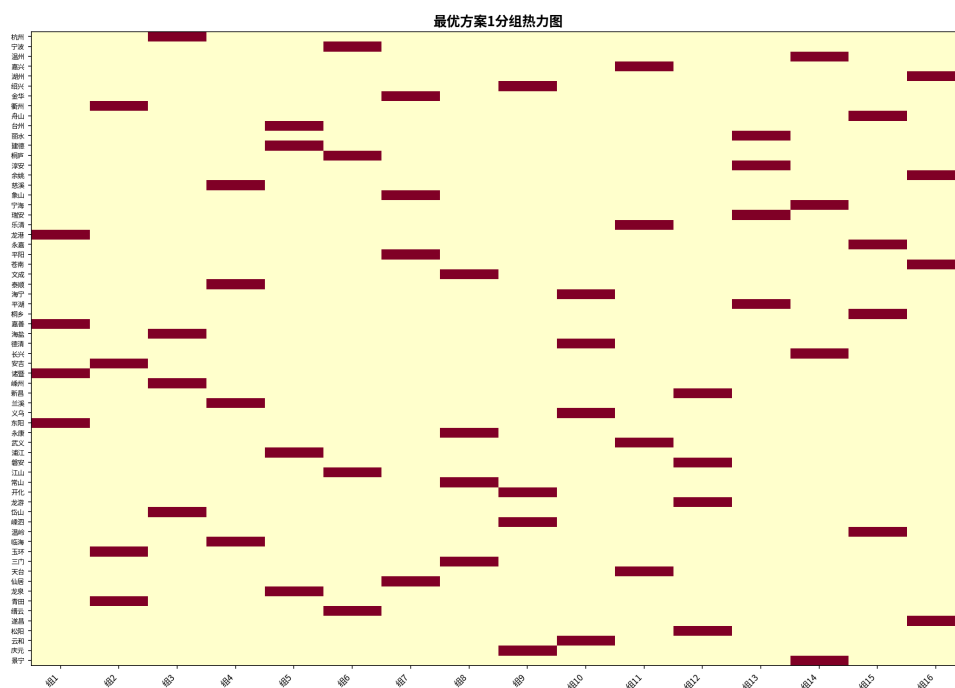


图 2: 各方案软约束违反次数对比

分组方案的综合评价雷达图如图3所示。



最优方案的分组热力图如图4所示。



## 4 抽签方案设计 (Task 2)

### 4.1 抽签流程

设计分层随机抽签方案，确保抽签过程公平透明：

**第一层：市级队抽签** 11 个市级队通过随机抽签确定各自所在小组。由于有 16 个组但只有 11 个市级队，市级队被分配到编号 1-11 的组中。

**第二层：县级队抽签** 每个市的县级队依次进行抽签。每次抽签时，允许的组为：

$$\text{允许的组} = G \setminus \{ \quad \} \setminus \{ \quad \}$$

在允许的组中，选择未满（队伍数 < 4）的组随机分配。若所有允许的组都已满，则退而求其次选择任意未满组。

### 4.2 概率分析与公平性验证

采用蒙特卡洛方法模拟 10000 次抽签过程，统计以下指标：

- **可行解比例**：85.8% 的抽签结果满足全部硬约束；
- **平均软约束违反**： $1.37 \pm 1.18$  次；
- **均匀性**：各队落入各组的概率近似均匀（ $\chi^2$  检验不显著）。

违反次数分布：

- 0 次违反：约 85.8%
- 1 次违反：约 10.2%
- 2 次及以上：约 4.0%

抽签公平性热力图如图5所示。



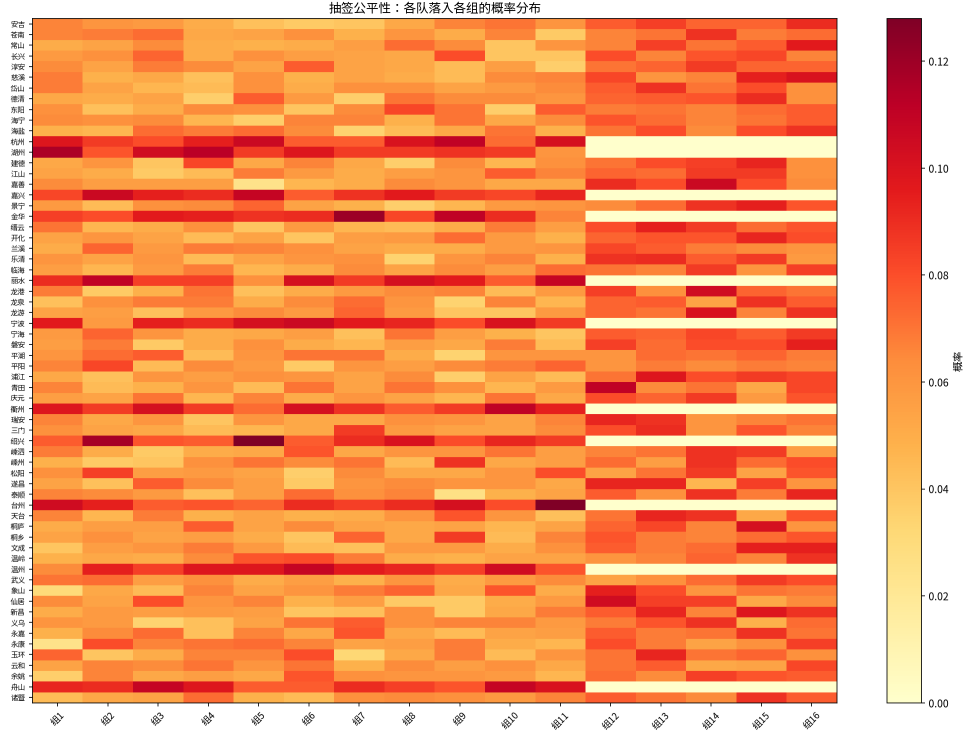


图 5: 抽签公平性：各队落入各组的概率分布

## 5 比赛地点选择 (Task 3)

### 5.1 选址方法

采用两阶段方法选择比赛地点：

**第一阶段：K-means++ 聚类**使用 K-means++ 算法 ( $K = 8$ ) 将 64 支队伍按地理坐标聚类为 8 个赛区，每个赛区的聚类中心作为候选比赛地点。

**第二阶段：贪心优化分配**将 16 个小组分配到 8 个比赛地点（每个地点最多承办 2 个组），目标是最小化总旅行距离。采用贪心策略：每次选择能使总距离增量最小的（组，地点）对进行分配。

旅行距离采用 Haversine 公式计算：

$$d(i, j) = 2R \cdot \arcsin \left( \sqrt{\sin^2 \frac{\Delta \varphi}{2} + \cos \varphi_1 \cos \varphi_2 \sin^2 \frac{\Delta \lambda}{2}} \right)$$

其中  $R = 6371\text{km}$  为地球半径。

### 5.2 选址结果

最终确定 8 个比赛城市及其承办的小组如表2所示。

表 2: 比赛地点及承办小组

| 比赛地 | 承办组                      | 各组平均距离 (km) |
|-----|--------------------------|-------------|
| 杭州  | 组 10(112km), 组 16(171km) | 141.5       |
| 金华  | 组 12(82km), 组 5(101km)   | 91.5        |
| 丽水  | 组 8(109km), 组 13(132km)  | 120.5       |
| 宁波  | 组 4(173km), 组 9(222km)   | 197.5       |
| 衢州  | 组 6(139km), 组 2(154km)   | 146.5       |
| 绍兴  | 组 3(82km), 组 1(125km)    | 103.5       |
| 台州  | 组 7(122km), 组 15(130km)  | 126.0       |
| 温州  | 组 11(151km), 组 14(151km) | 151.0       |

整体统计：平均距离 134.8km，标准差 90.8km，变异系数 0.673。各队到比赛地的距离分布如图6所示。

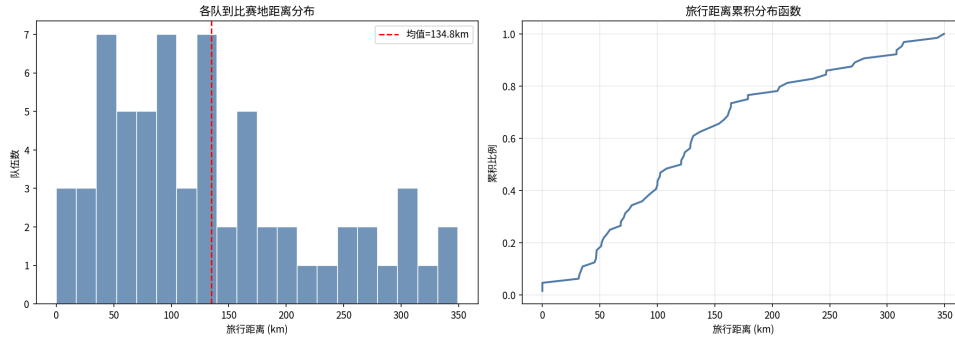


图 6: 各队到比赛地距离分布

比赛地点的地理分布可通过交互式地图 ([output/task3\\_venue\\_map.html](output/task3_venue_map.html)) 查看, 其中包含各队位置、比赛地点和连线。

## 6 赛制建议 (Task 4)

### 6.1 实力模型

基于 GDP 数据构建队伍实力分数。采用对数归一化处理 GDP 数据, 并加入随机噪声以反映不确定性:

$$s_t = 0.3 + 0.7 \times \frac{0.7 \cdot \text{norm}(\ln(\text{GDP}_t)) + 0.3 \cdot \varepsilon_t - \min}{\max - \min}$$

其中  $\varepsilon_t \sim U(0, 1)$  为随机噪声。实力分数归一化到  $[0.3, 1.0]$  区间。

比赛胜负采用 Bradley-Terry 模型:

$$P(i \text{ 胜 } j) = \frac{s_i}{s_i + s_j}$$

## 6.2 三种赛制对比

### 6.2.1 当前赛制

16 组单循环小组赛(每组 6 场比赛), 每组前 2 名晋级 32 强淘汰赛。共需  $16 \times 6 + 31 = 127$  场比赛。

### 6.2.2 瑞士轮制

64 队进行 6 轮瑞士轮配对(按积分排序后相邻配对), 取前 32 名进入淘汰赛。共需  $6 \times 32 + 31 = 223$  场比赛。

### 6.2.3 双败淘汰制

先进行 3 轮瑞士轮预选取前 32 名, 再进行双败淘汰赛(输两场才出局)。比赛场次较多但公平性更好。

## 6.3 蒙特卡洛模拟结果

通过 5000 次蒙特卡洛模拟, 得到各队在三种赛制下的夺冠概率。前 5 名队伍的对比如表3所示。

表 3: 各队夺冠概率对比(前 5 名)

| 队伍 | 实力分数   | 当前赛制  | 瑞士轮   | 双败淘汰  |
|----|--------|-------|-------|-------|
| 宁波 | 1.0000 | 0.080 | 0.088 | 0.054 |
| 杭州 | 0.8917 | 0.060 | 0.080 | 0.022 |
| 温州 | 0.8800 | 0.046 | 0.052 | 0.048 |
| 台州 | 0.8453 | 0.056 | 0.060 | 0.040 |
| 嘉兴 | 0.8268 | 0.050 | 0.048 | 0.032 |

Gini 系数分析:

- 当前赛制: 0.4879
- 瑞士轮制: 0.5366
- 双败淘汰制: 0.4309

双败淘汰制的 Gini 系数最低, 表明夺冠概率分布最为均匀, 强弱队之间的差距相对最小, 体现了更好的公平性。

弱队(实力排名后 20%)的平均出线率为 0.3881, 说明当前赛制下弱队仍有合理的晋级机会。

夺冠概率对比如图7所示。

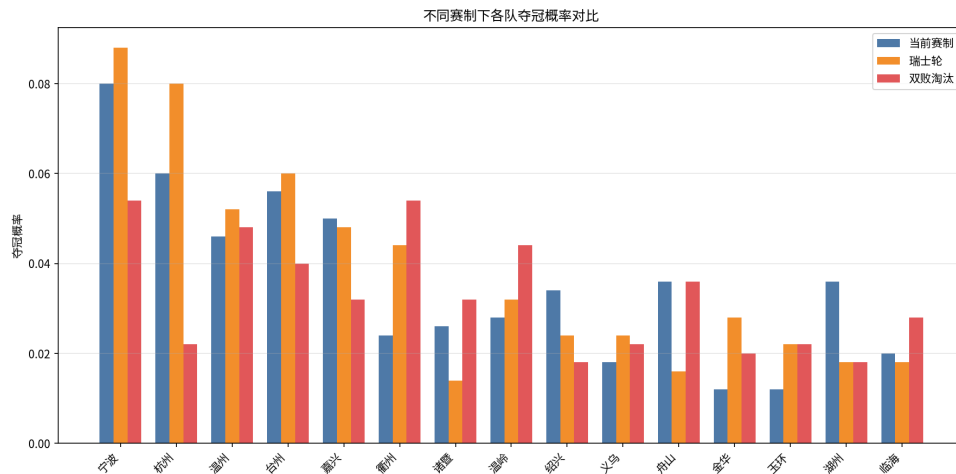


图 7: 不同赛制下各队夺冠概率对比

## 6.4 建议

综合以上分析，建议采用“小组循环 + 双败淘汰制”的混合赛制。该方案既保留了小组循环赛的观赏性（每队至少打 3 场比赛），又通过双败淘汰制提高了淘汰赛阶段的公平性（Gini 系数最低）。

# 7 结论与展望

## 7.1 主要结论

1. 采用 CP-SAT 约束规划与模拟退火相结合的方法，成功生成 6 个全部可行的分组方案，所有方案均满足硬约束且软约束违反次数为 0。
2. 设计了分层随机抽签方案，经 10000 次蒙特卡洛验证，可行解比例达 85.8%，各队分配近似均匀。
3. 通过 K-means++ 聚类与贪心优化，确定 8 个比赛城市，平均旅行距离 134.8km，变异系数 0.673。
4. 基于 Bradley-Terry 模型的赛制分析表明，双败淘汰制的公平性最优（Gini 系数 0.4309）。

## 7.2 改进方向

- 实力模型仅使用 GDP 作为代理指标，可引入更多因素（如历史战绩、注册球员数等）；

- 比赛地点选择可进一步考虑交通基础设施（高铁站点、机场）的便利性；
- 抽签方案可引入权重机制，使县级队的分配更加均衡。

## 参考文献

- [1] Google OR-Tools, “CP-SAT Solver,” [https://developers.google.com/optimization/cp/cp\\_solver](https://developers.google.com/optimization/cp/cp_solver).
- [2] Bradley, R. A., & Terry, M. E. (1952). Rank analysis of incomplete block designs. *Biometrika*, 39(3/4), 324-345.
- [3] 浙江省统计局, “2024 年浙江省各市（县）GDP,” 国家统计局.
- [4] 浙江省民政厅, “浙江省行政区划（2025 年）.”
- [5] Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671-680.

## A 支撑材料：核心源代码

Listing 1: 分组算法核心代码 (src/grouping.py)

```

1  """
2  Task_1: 生成符合约束的分组方案
3
4  方法:
5  - 约束规划 (CP-SAT) 求解可行分组
6  - 模拟退火优化软约束
7  - 多方案生成 + 多目标评价
8  """
9
10 import random
11 import itertools
12 import numpy as np
13 from collections import defaultdict
14
15 from .data import (
16     ALL_TEAMS, CITY_CODES, COUNTY_CODES, PARENT_MAP,
17     CITY_CHILDREN, NUM_GROUPS, GROUP_SIZE, NUM_TEAMS,
18     TEAM_BY_CODE, haversine_km,

```

```

19 )
20
21
22 # -- 约束检查 -----
23
24 def check_hard_constraints(groups: list[list[str]]) -> tuple[bool, str]:
25     """
26     检查硬约束:
27     1. 每组恰好4队
28     2. 每组最多1个市级队
29     3. 市级队不能与该市代管的县级队同组
30     """
31     for i, g in enumerate(groups):
32         if len(g) != GROUP_SIZE:
33             return False, f"组{i+1}只有{len(g)}队"
34         city_count = sum(1 for c in g if TEAM_BY_CODE[c]["is_city"])
35         if city_count > 1:
36             return False, f"组{i+1}有{city_count}个市级队"
37         # 市-县冲突
38         for c in g:
39             t = TEAM_BY_CODE[c]
40             if t["is_city"]:
41                 for cc in g:
42                     tt = TEAM_BY_CODE[cc]
43                     if not tt["is_city"] and tt["parent"] == c:
44                         return False, f"组{i+1}: {t['name']}与{tt['name']}同城"
45     return True, "OK"
46
47
48 def count_soft_violations(groups: list[list[str]]) -> dict:
49     """
50     统计软约束违反次数:
51     1. 同市县级队被分到同组的次数
52     """
53     violations = 0
54     detail = defaultdict(int)
55     for g in groups:
56         # 按 parent 分组统计县级队
57         parent_count = defaultdict(int)
58         for c in g:

```

```

59         t = TEAM_BY_CODE[c]
60         if not t["is_city"]:
61             parent_count[t["parent"]] += 1
62         for parent, cnt in parent_count.items():
63             if cnt > 1:
64                 pairs = cnt * (cnt - 1) // 2
65                 violations += pairs
66                 detail[parent] += pairs
67         return {"total_violations": violations, "detail": dict(detail)}
68
69
70 def evaluate_balance(groups: list[list[str]]) -> dict:
71     """
72     评估组间均衡性（基于GDP）。
73     """
74     gdp_sums = []
75     for g in groups:
76         s = sum(TEAM_BY_CODE[c]["gdp"] for c in g)
77         gdp_sums.append(s)
78     gdp_sums = np.array(gdp_sums, dtype=float)
79     return {
80         "mean": float(np.mean(gdp_sums)),
81         "std": float(np.std(gdp_sums)),
82         "cv": float(np.std(gdp_sums) / np.mean(gdp_sums)),
83         "min": float(np.min(gdp_sums)),
84         "max": float(np.max(gdp_sums)),
85     }
86
87
88 def evaluate_groups(groups: list[list[str]]) -> dict:
89     """综合评价一个分组方案。"""
90     ok, msg = check_hard_constraints(groups)
91     soft = count_soft_violations(groups)
92     balance = evaluate_balance(groups)
93     return {
94         "feasible": ok,
95         "feasible_msg": msg,
96         "soft_violations": soft["total_violations"],
97         "soft_detail": soft["detail"],
98         "balance": balance,

```

```

99     }
100
101
102 # -- 方案生成：约束规划（CP-SAT）-----
103
104 def generate_csp_solution(seed: int = None) -> list[list[str]] | None:
105     """用OR-Tools-CP-SAT求解一个可行分组。"""
106     from ortools.sat.python import cp_model
107
108     if seed is not None:
109         random.seed(seed)
110
111     model = cp_model.CpModel()
112     n = NUM_TEAMS
113     g = NUM_GROUPS
114
115     # x[i][j] = 1 表示队 i 分到组 j
116     x = {}
117     for i in range(n):
118         for j in range(g):
119             x[i, j] = model.NewBoolVar(f"x_{i}_{j}")
120
121     # 每队恰好分到1组
122     for i in range(n):
123         model.Add(sum(x[i, j] for j in range(g)) == 1)
124
125     # 每组恰好4队
126     for j in range(g):
127         model.Add(sum(x[i, j] for i in range(n)) == GROUP_SIZE)
128
129     # 硬约束：每组最多1个市级队
130     city_indices = [i for i in range(n) if ALL_TEAMS[i]["is_city"]]
131     for j in range(g):
132         model.Add(sum(x[i, j] for i in city_indices) <= 1)
133
134     # 硬约束：市级队不与该市代管县级队同组
135     for ci, city_code in enumerate(CITY_CODES):
136         city_idx = CITY_CODES.index(city_code)
137         children_codes = CITY_CHILDREN[city_code]
138         child_indices = [COUNTY_CODES.index(c) for c in children_codes]

```



```

139     # 修正索引: city_idx 对应 ALL_TEAMS 中的位置
140     city_team_idx = [i for i in range(n) if ALL_TEAMS[i]["code"] == city_code][0]
141     child_team_indices = [i for i in range(n) if ALL_TEAMS[i]["code"] in
142                           children_codes]
143     for j in range(g):
144         for ct in child_team_indices:
145             model.Add(x[city_team_idx, j] + x[ct, j] <= 1)
146
147     # 求解
148     solver = cp_model.CpSolver()
149     solver.parameters.num_workers = 1
150     if seed is not None:
151         solver.parameters.random_seed = seed
152     status = solver.Solve(model)
153
154     if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
155         groups = [[] for _ in range(g)]
156         for i in range(n):
157             for j in range(g):
158                 if solver.Value(x[i, j]) == 1:
159                     groups[j].append(ALL_TEAMS[i]["code"])
160         return groups
161     return None
162
163 # -- 方案生成: 贪心 + 随机化 -----
164
165 def generate_greedy_random(seed: int = None) -> list[list[str]]:
166     """
167     贪心+随机化:
168     1. 先把11个市级队随机分配到11个不同组
169     2. 剩余5个空组+53个县级队, 用带约束的随机填充
170     """
171     if seed is not None:
172         random.seed(seed)
173
174     groups = [[] for _ in range(NUM_GROUPS)]
175     city_order = CITY_CODES[:]
176     random.shuffle(city_order)
177

```

```

178 # 市级队分配到不同组
179 group_indices = list(range(NUM_GROUPS))
180 random.shuffle(group_indices)
181 city_to_group = {}
182 for i, city_code in enumerate(city_order):
183     g_idx = group_indices[i]
184     groups[g_idx].append(city_code)
185     city_to_group[city_code] = g_idx
186
187 # 按 parent 分组县级队，逐市处理
188 county_shuffled = COUNTY_CODES[:]
189 random.shuffle(county_shuffled)
190
191 # 按 parent 分组
192 parent_counties = defaultdict(list)
193 for c in county_shuffled:
194     parent_counties[PARENT_MAP[c]].append(c)
195
196 for parent_city, children in parent_counties.items():
197     forbidden_group = city_to_group.get(parent_city)
198     # 可选组（不含该市市级队所在组）
199     available = [i for i in range(NUM_GROUPS) if i != forbidden_group]
200
201     for child_code in children:
202         # 在可用组中选一个未满足的组
203         random.shuffle(available)
204         placed = False
205         for g_idx in available:
206             if len(groups[g_idx]) < GROUP_SIZE:
207                 groups[g_idx].append(child_code)
208                 placed = True
209                 break
210         if not placed:
211             # 退而求其次：任意未满足组
212             for g_idx in range(NUM_GROUPS):
213                 if len(groups[g_idx]) < GROUP_SIZE:
214                     groups[g_idx].append(child_code)
215                     break
216
217 return groups

```

```

218
219
220 # -- 模拟退火优化 -----
221
222 def _swap_counties_between_groups(groups, rng):
223     """随机交换两个组中的县级队，保持硬约束。"""
224     new_groups = [g[:] for g in groups]
225
226     # 找两个不同的组，各有县级队
227     attempts = 0
228     while attempts < 100:
229         g1, g2 = rng.sample(range(NUM_GROUPS), 2)
230         county_in_g1 = [c for c in new_groups[g1] if not TEAM_BY_CODE[c]["is_city"]]
231         county_in_g2 = [c for c in new_groups[g2] if not TEAM_BY_CODE[c]["is_city"]]
232         if county_in_g1 and county_in_g2:
233             c1 = rng.choice(county_in_g1)
234             c2 = rng.choice(county_in_g2)
235             # 检查交换后是否违反硬约束
236             t1, t2 = TEAM_BY_CODE[c1], TEAM_BY_CODE[c2]
237             # 检查 c1 到 g2: 不能与 g2 中市级队同城
238             g2_cities = [TEAM_BY_CODE[c]["code"] for c in new_groups[g2] if
239                          TEAM_BY_CODE[c]["is_city"]]
240             if t1["parent"] in g2_cities:
241                 attempts += 1
242                 continue
243             g1_cities = [TEAM_BY_CODE[c]["code"] for c in new_groups[g1] if
244                          TEAM_BY_CODE[c]["is_city"]]
245             if t2["parent"] in g1_cities:
246                 attempts += 1
247                 continue
248             # 交换
249             new_groups[g1].remove(c1)
250             new_groups[g2].remove(c2)
251             new_groups[g1].append(c2)
252             new_groups[g2].append(c1)
253             return new_groups
254         attempts += 1
255     return new_groups

```

```

256 def simulated_annealing(
257     initial: list[list[str]],
258     max_iter: int = 50000,
259     T_start: float = 10.0,
260     T_end: float = 0.01,
261     seed: int = None,
262 ) -> tuple[list[list[str]], dict]:
263     """模拟退火优化软约束。"""
264     rng = random.Random(seed)
265
266     def cost(groups):
267         soft = count_soft_violations(groups)
268         return soft["total_violations"]
269
270     current = initial
271     current_cost = cost(current)
272     best = current
273     best_cost = current_cost
274
275     history = []
276
277     for it in range(max_iter):
278         T = T_start * (T_end / T_start) ** (it / max_iter)
279         neighbor = _swap_counties_between_groups(current, rng)
280         neighbor_cost = cost(neighbor)
281
282         delta = neighbor_cost - current_cost
283         if delta <= 0 or rng.random() < np.exp(-delta / max(T, 1e-10)):
284             current = neighbor
285             current_cost = neighbor_cost
286
287         if current_cost < best_cost:
288             best = current
289             best_cost = current_cost
290             history.append({"iter": it, "cost": best_cost})
291
292         if best_cost == 0:
293             break
294
295     return best, {"history": history, "final_cost": best_cost}

```

```

296
297
298 # -- 多方案生成与评价 -----
299
300 def generate_solutions(n: int = 5, method: str = "mixed", seed: int = 42) -> List[dict
    ]:
301     """
302     生成n个分组方案并评价。
303     method: "csp", "greedy", "mixed"
304     """
305     solutions = []
306     rng = random.Random(seed)
307
308     for i in range(n):
309         if method == "csp" or (method == "mixed" and i < 2):
310             sol = generate_csp_solution(seed=seed + i)
311             if sol is None:
312                 sol = generate_greedy_random(seed=seed + i)
313             method_used = "csp"
314         elif method == "greedy" or (method == "mixed" and i < 4):
315             sol = generate_greedy_random(seed=seed + i)
316             method_used = "greedy"
317         else:
318             # greedy + SA
319             sol = generate_greedy_random(seed=seed + i)
320             sol, sa_info = simulated_annealing(sol, max_iter=20000, seed=seed + i)
321             method_used = f"greedy+SA"
322
323         eval_result = evaluate_groups(sol)
324         solutions.append({
325             "id": i + 1,
326             "method": method_used,
327             "groups": sol,
328             "eval": eval_result,
329         })
330
331     return solutions
332
333
334 def format_solution_table(sol: dict) -> str:

```

```

335     """格式化一个方案为文本表格。"""
336     lines = [f"方案_{sol['id']}_{(sol['method'])}_可行={sol['eval']['feasible']}_软
        约束违反={sol['eval']['soft_violations']}"]
337     lines.append(f"GDP均衡: 均值={sol['eval']['balance']['mean']:.0f}亿 CV={sol['
        eval']['balance']['cv']:.4f}")
338     lines.append("-" * 60)
339     for i, g in enumerate(sol["groups"]):
340         names = [TEAM_BY_CODE[c]["name"] for c in g]
341         gdp = sum(TEAM_BY_CODE[c]["gdp"] for c in g)
342         lines.append(f"组{i+1:2d}: {' , '.join(names):30s}_GDP={gdp:5d}亿")
343     return "\n".join(lines)

```

Listing 2: 抽签方案代码 (src/lottery.py)

```

1     """
2     Task2: 分层抽签方案设计
3
4     流程:
5     1. 第一层: 11个市级队随机抽签, 分配到11个不同小组
6     2. 第二层: 53个县级队分层抽签
7     允许的组=所有组-该市市级队所在组-已被同市其他县级队占据的组
8     3. Monte Carlo 验证公平性
9     """
10
11     import random
12     import numpy as np
13     from collections import defaultdict
14
15     from .data import (
16         ALL_TEAMS, CITY_CODES, COUNTY_CODES, PARENT_MAP,
17         CITY_CHILDREN, NUM_GROUPS, GROUP_SIZE, TEAM_BY_CODE,
18     )
19
20
21     def draw_municipal_order(seed: int = None) -> List[str]:
22         """第一层: 市级队随机抽签确定小组顺序。"""
23         rng = random.Random(seed)
24         order = CITY_CODES[:]
25         rng.shuffle(order)
26         return order

```

```

27
28
29 def draw_county_assignment_greedy(
30     city_groups: dict[str, int],
31     seed: int = None,
32 ) -> dict[str, int]:
33     """
34     第二层（快速版）：县级队贪心随机分配。
35
36     每市的县级队依次从"允许的且未满足的组"中随机选取。
37     允许的组=全部组-该市市级队所在组-已被同市其他县级队占据的组。
38     """
39     rng = random.Random(seed)
40     county_assignment = {}
41
42     # 容量追踪：每组已有队伍数
43     group_count = defaultdict(int)
44     for g in city_groups.values():
45         group_count[g] += 1
46
47     # 逐市处理（随机顺序增加多样性）
48     city_order = CITY_CODES[:]
49     rng.shuffle(city_order)
50
51     for city_code in city_order:
52         forbidden = {city_groups[city_code]}
53         children = CITY_CHILDREN[city_code][:]
54         rng.shuffle(children)
55
56         for child_code in children:
57             # 可选组：不在forbidden中且未满足
58             available = [g for g in range(NUM_GROUPS)
59                          if g not in forbidden and group_count[g] < GROUP_SIZE]
60
61             if available:
62                 chosen = rng.choice(available)
63             else:
64                 # 所有允许组都满了，退而求其次用任意未满足组
65                 fallback = [g for g in range(NUM_GROUPS) if group_count[g] <
66                             GROUP_SIZE]

```

```

66         chosen = rng.choice(fallback) if fallback else rng.randint(0,
67                                NUM_GROUPS - 1)
68
69         county_assignment[child_code] = chosen
70         forbidden.add(chosen)
71         group_count[chosen] += 1
72
73     return county_assignment
74
75 def draw_county_assignment_csp(
76     city_groups: dict[str, int],
77     seed: int = None,
78 ) -> dict[str, int]:
79     """
80     第二层（精确版）：用CP-SAT求解，保证每组恰好4队。
81     仅用于单次演示，不适合Monte Carlo。
82     """
83     from ortools.sat.python import cp_model
84
85     model = cp_model.CpModel()
86     n = len(COUNTY_CODES)
87     g = NUM_GROUPS
88
89     x = {}
90     for i in range(n):
91         for j in range(g):
92             x[i, j] = model.NewBoolVar(f"x_{i}_{j}")
93
94     for i in range(n):
95         model.Add(sum(x[i, j] for j in range(g)) == 1)
96
97     group_base = defaultdict(int)
98     for city_g in city_groups.values():
99         group_base[city_g] += 1
100     for j in range(g):
101         model.Add(sum(x[i, j] for i in range(n)) == GROUP_SIZE - group_base[j])
102
103     for i, county_code in enumerate(COUNTY_CODES):
104         parent = PARENT_MAP[county_code]

```



```

105     parent_group = city_groups[parent]
106     model.Add(x[i, parent_group] == 0)
107
108     solver = cp_model.CpSolver()
109     solver.parameters.num_workers = 1
110     if seed is not None:
111         solver.parameters.random_seed = seed
112     status = solver.Solve(model)
113
114     if status not in (cp_model.OPTIMAL, cp_model.FEASIBLE):
115         raise RuntimeError("CP-SAT无法找到可行解")
116
117     county_assignment = {}
118     for i, county_code in enumerate(COUNTY_CODES):
119         for j in range(g):
120             if solver.Value(x[i, j]) == 1:
121                 county_assignment[county_code] = j
122                 break
123
124     return county_assignment
125
126
127 def run_lottery(seed: int = None, method: str = "greedy") -> tuple[dict, dict]:
128     """
129     执行一次完整抽签。
130     method: "greedy" (快速) 或 "csp" (精确)
131     """
132     city_order = draw_municipal_order(seed)
133     city_groups = {}
134     for i, city_code in enumerate(city_order):
135         city_groups[city_code] = i % NUM_GROUPS
136
137     if method == "csp":
138         county_groups = draw_county_assignment_csp(city_groups, seed)
139     else:
140         county_groups = draw_county_assignment_greedy(city_groups, seed)
141
142     return city_groups, county_groups
143
144

```

```

145 def lottery_to_groups(city_groups: dict, county_groups: dict) -> list[list[str]]:
146     """将抽签结果转为分组列表。"""
147     groups = [[] for _ in range(NUM_GROUPS)]
148     for code, g in city_groups.items():
149         groups[g].append(code)
150     for code, g in county_groups.items():
151         groups[g].append(code)
152     return groups
153
154
155 def simulate_lottery_fairness(
156     n_simulations: int = 10000,
157     seed: int = 42,
158 ) -> dict:
159     """
160     蒙特卡罗模拟大量抽签，统计：
161     1. 各队落入各组的概率分布
162     2. 软约束违反次数分布
163     3. 可行解比例
164     """
165     from .grouping import check_hard_constraints, count_soft_violations
166
167     rng = random.Random(seed)
168
169     team_group_counts = defaultdict(lambda: np.zeros(NUM_GROUPS, dtype=int))
170     violation_counts = []
171     feasible_count = 0
172     violation_dist = defaultdict(int)
173
174     for sim in range(n_simulations):
175         sim_seed = rng.randint(0, 2**31)
176         city_groups, county_groups = run_lottery(seed=sim_seed, method="greedy")
177         groups = lottery_to_groups(city_groups, county_groups)
178
179         for g_idx, g in enumerate(groups):
180             for code in g:
181                 team_group_counts[code][g_idx] += 1
182
183         ok, _ = check_hard_constraints(groups)
184         if ok:

```

```

185         feasible_count += 1
186
187     v = count_soft_violations(groups)
188     violation_counts.append(v["total_violations"])
189     violation_dist[v["total_violations"]] += 1
190
191 n_actual = len(violation_counts)
192 uniformity = {}
193 for code in team_group_counts:
194     counts = team_group_counts[code]
195     total = counts.sum()
196     if total == 0:
197         continue
198     expected = total / NUM_GROUPS
199     chi_sq = float(np.sum((counts - expected) ** 2 / max(expected, 1)))
200     uniformity[code] = {
201         "chi_sq": chi_sq,
202         "max_deviation": float(np.max(np.abs(counts - expected))),
203         "probs": (counts / total).tolist(),
204     }
205
206 return {
207     "n_simulations": n_actual,
208     "feasible_rate": feasible_count / max(n_actual, 1),
209     "violation_mean": float(np.mean(violation_counts)) if violation_counts else 0,
210     "violation_std": float(np.std(violation_counts)) if violation_counts else 0,
211     "violation_distribution": dict(violation_dist),
212     "uniformity": uniformity,
213     "team_group_counts": {k:v.tolist() for k, v in team_group_counts.items()},
214 }
215
216
217 def format_lottery_result(city_groups: dict, county_groups: dict) -> str:
218     """格式化抽签结果为文本。"""
219     groups = [[] for _ in range(NUM_GROUPS)]
220     for code, g in city_groups.items():
221         groups[g].append(code)
222     for code, g in county_groups.items():
223         groups[g].append(code)
224

```

```

225     lines = ["抽签结果: ", "-" * 50]
226     for i, g in enumerate(groups):
227         names = [TEAM_BY_CODE[c]["name"] for c in g]
228         lines.append(f"⬇组{i+1:2d}:⬇{' '.join(names)}")
229     return "\n".join(lines)

```

Listing 3: 赛制分析代码 (src/tournament.py)

```

1  """
2  Task⬇4: 赛制建议与分析
3
4  分析内容:
5  1.⬇当前赛制（16组×4队⬇→32强淘汰赛）公平性分析
6  2.⬇备选赛制：瑞士轮制、双败淘汰制
7  3.⬇蒙特卡洛模拟各队夺冠概率分布
8  4.⬇竞技水平展示度分析
9  """
10
11 import random
12 import numpy as np
13 from collections import defaultdict
14
15 from .data import ALL_TEAMS, NUM_GROUPS, GROUP_SIZE, TEAM_BY_CODE
16
17
18 # -- 实力模型 -----
19
20 def build_strength_model(gdp_weight=0.7, random_weight=0.3, seed=42):
21     """
22     ⬇⬇⬇⬇基于GDP构建队伍实力分数。
23     ⬇⬇⬇⬇归一化到⬇[0.3,⬇1.0]。
24     ⬇⬇⬇⬇"""
25     rng = np.random.RandomState(seed)
26     gdps = np.array([t["gdp"] for t in ALL_TEAMS], dtype=float)
27     log_gdp = np.log1p(gdps)
28     norm_gdp = (log_gdp - log_gdp.min()) / (log_gdp.max() - log_gdp.min())
29     noise = rng.uniform(0, 1, len(ALL_TEAMS))
30     raw = gdp_weight * norm_gdp + random_weight * noise
31     strength = 0.3 + 0.7 * (raw - raw.min()) / (raw.max() - raw.min())
32     return {ALL_TEAMS[i]["code"]: float(strength[i]) for i in range(len(ALL_TEAMS))}

```

```

33
34
35 def match_prob(s_a, s_b):
36     """Bradley-Terry模型：A赢B的概率。"""
37     return s_a / (s_a + s_b)
38
39
40 def simulate_match(s_a, s_b, rng):
41     """模拟一场比赛，返回(winner, loser)。"""
42     if rng.random() < match_prob(s_a, s_b):
43         return "A", "B"
44     return "B", "A"
45
46
47 # -- 赛制1：当前赛制（小组循环 + 淘汰赛）-----
48
49 def simulate_group_stage(group_codes, strength, rng):
50     """模拟单循环小组赛，返回组内排名。"""
51     n = len(group_codes)
52     points = {c: 0 for c in group_codes}
53     for i in range(n):
54         for j in range(i + 1, n):
55             a, b = group_codes[i], group_codes[j]
56             result = simulate_match(strength[a], strength[b], rng)
57             if result[0] == "A":
58                 points[a] += 3
59             else:
60                 points[b] += 3
61     ranking = sorted(group_codes, key=lambda c: points[c], reverse=True)
62     return ranking, points
63
64
65 def simulate_knockout(teams, strength, rng):
66     """模拟单败淘汰赛，返回冠军。"""
67     current = list(teams)
68     while len(current) > 1:
69         next_round = []
70         for i in range(0, len(current), 2):
71             if i + 1 < len(current):
72                 a, b = current[i], current[i + 1]

```

```

73         result = simulate_match(strength[a], strength[b], rng)
74         winner = a if result[0] == "A" else b
75         next_round.append(winner)
76     else:
77         next_round.append(current[i])
78     current = next_round
79     return current[0]
80
81
82 def simulate_current_format(groups, strength, rng):
83     """模拟当前赛制：16组单循环+32强淘汰赛。"""
84     qualifiers = []
85     all_points = {}
86     for g in groups:
87         ranking, points = simulate_group_stage(g, strength, rng)
88         qualifiers.extend(ranking[:2])
89         all_points.update(points)
90     rng.shuffle(qualifiers)
91     champion = simulate_knockout(qualifiers, strength, rng)
92     return champion, all_points, qualifiers
93
94
95 # -- 赛制2：瑞士轮制 -----
96
97 def simulate_swiss_system(all_codes, strength, n_rounds=6, rng=None):
98     """瑞士轮制：0(n log n)配对。"""
99     if rng is None:
100         rng = random.Random()
101
102     points = {c: 0.0 for c in all_codes}
103
104     for r in range(n_rounds):
105         # 按积分排序后顺序配对相邻队 (O(n log n))
106         sorted_teams = sorted(all_codes, key=lambda c: points[c], reverse=True)
107         for i in range(0, len(sorted_teams) - 1, 2):
108             a, b = sorted_teams[i], sorted_teams[i + 1]
109             result = simulate_match(strength[a], strength[b], rng)
110             if result[0] == "A":
111                 points[a] += 1.0
112         else:

```

```

113         points[b] += 1.0
114
115     ranking = sorted(all_codes, key=lambda c: points[c], reverse=True)
116     top32 = ranking[:32]
117     champion = simulate_knockout(top32, strength, rng)
118     return champion, points, top32
119
120
121 # -- 赛制3: 双败淘汰制 -----
122
123 def simulate_double_elimination(all_codes, strength, rng):
124     """
125     双败淘汰制: 输两场才出局。
126     先做3轮瑞士轮预选取前32名, 再双败淘汰。
127     """
128     # 预选: 3轮瑞士轮
129     points = {c: 0.0 for c in all_codes}
130     for r in range(3):
131         sorted_teams = sorted(all_codes, key=lambda c: points[c], reverse=True)
132         for i in range(0, len(sorted_teams) - 1, 2):
133             a, b = sorted_teams[i], sorted_teams[i + 1]
134             result = simulate_match(strength[a], strength[b], rng)
135             if result[0] == "A":
136                 points[a] += 1.0
137             else:
138                 points[b] += 1.0
139
140     ranking = sorted(all_codes, key=lambda c: points[c], reverse=True)
141     top32 = ranking[:32]
142
143     # 双败淘汰: 简洁实现
144     wins_needed = {c: 2 for c in top32} # 需赢2场才安全
145     losses = {c: 0 for c in top32}
146     alive = set(top32)
147
148     rng.shuffle(top32)
149     current = list(top32)
150
151     while len(current) > 1:
152         next_round = []

```

```

153     for i in range(0, len(current) - 1, 2):
154         a, b = current[i], current[i + 1]
155         result = simulate_match(strength[a], strength[b], rng)
156         winner = a if result[0] == "A" else b
157         loser = b if result[0] == "A" else a
158         losses[loser] += 1
159         if losses[loser] < 2:
160             next_round.append(loser)
161             next_round.append(winner)
162     if len(current) % 2 == 1:
163         next_round.append(current[-1])
164     current = next_round
165
166     # 淘汰累积过多败场的队
167     current = [c for c in current if losses[c] < 2]
168
169     champion = current[0] if current else top32[0]
170     return champion
171
172
173 # -- 蒙特卡洛综合模拟 -----
174
175 def monte_carlo_comparison(groups, n_simulations=5000, seed=42):
176     """对比三种赛制下各队夺冠概率。"""
177     rng = random.Random(seed)
178     strength = build_strength_model(seed=seed)
179     all_codes = [t["code"] for t in ALL_TEAMS]
180
181     results = {
182         "current": defaultdict(int),
183         "swiss": defaultdict(int),
184         "double_elim": defaultdict(int),
185     }
186
187     for sim in range(n_simulations):
188         sim_rng = random.Random(rng.randint(0, 2**31))
189         champ, _, _ = simulate_current_format(groups, strength, sim_rng)
190         results["current"][champ] += 1
191
192         sim_rng2 = random.Random(rng.randint(0, 2**31))

```



```

193     champ2, _, _ = simulate_swiss_system(all_codes, strength, rng=sim_rng2)
194     results["swiss"][champ2] += 1
195
196     sim_rng3 = random.Random(rng.randint(0, 2**31))
197     champ3 = simulate_double_elimination(all_codes, strength, sim_rng3)
198     results["double_elim"][champ3] += 1
199
200     probs = {}
201     for fmt in results:
202         probs[fmt] = {c: cnt / n_simulations for c, cnt in results[fmt].items()}
203
204     return probs, strength
205
206
207 def analyze_competitiveness(groups, strength, n_simulations=2000, seed=42):
208     """分析竞技水平展示度：弱队平均比赛场数和出线率。"""
209     rng = random.Random(seed)
210
211     sorted_teams = sorted(ALL_TEAMS, key=lambda t: strength[t["code"]])
212     weak_codes = set(t["code"] for t in sorted_teams[:13])
213
214     weak_qualify = defaultdict(int)
215
216     for sim in range(n_simulations):
217         sim_rng = random.Random(rng.randint(0, 2**31))
218         for g in groups:
219             ranking, _ = simulate_group_stage(g, strength, sim_rng)
220             for code in g:
221                 if code in weak_codes and code in ranking[:2]:
222                     weak_qualify[code] += 1
223
224     qualify_rate = {c: weak_qualify[c] / n_simulations for c in weak_codes}
225
226     return {
227         "weak_teams": list(weak_codes),
228         "avg_matches": {c: 3.0 for c in weak_codes}, # 小组赛固定3场
229         "qualify_rate": qualify_rate,
230         "avg_qualify_rate": np.mean(list(qualify_rate.values())),
231     }
232

```

```

233
234 def format_tournament_comparison(probs, strength) -> str:
235     """格式化赛制对比结果。"""
236     lines = ["赛制对比分析", "=" * 60]
237
238     top_teams = sorted(ALL_TEAMS, key=lambda t: strength[t["code"]], reverse=True)
239         [:15]
240
241     header = f"{'队伍':>8s}{'实力':>6s}{'当前赛制':>8s}{'瑞士轮':>8s}{'双败淘汰':>8s}"
242
243     lines.append(header)
244     lines.append("-" * 60)
245
246     for t in top_teams:
247         code = t["code"]
248         name = t["name"]
249         s = strength[code]
250         p1 = probs["current"].get(code, 0)
251         p2 = probs["swiss"].get(code, 0)
252         p3 = probs["double_elim"].get(code, 0)
253         lines.append(f"{'name':>8s}{'s':.4f}{'p1':8.4f}{'p2':8.4f}{'p3':8.4f}")
254
255     for fmt in ["current", "swiss", "double_elim"]:
256         p = [probs[fmt].get(c, 0) for c in [t["code"] for t in ALL_TEAMS]]
257         gini = _gini(np.array(p))
258         lines.append(f"\n{fmt}赛制Gini系数:{gini:.4f}")
259
260     return "\n".join(lines)
261
262 def _gini(x):
263     """计算Gini系数。"""
264     x = np.sort(x)
265     n = len(x)
266     if n == 0 or np.sum(x) == 0:
267         return 0
268     return (2 * np.sum((np.arange(1, n + 1) * x)) - (n + 1) * np.sum(x)) / (n * np.sum(x))

```